Proyecto Final

Este Notebook es el punto de partida para el Proyecto Final del curso de Deep Learning, este proyecto trata sobre un fenómeno conocido como el "Lottery Ticket Hypothesis".

01 - MNIST MLP

Este Notebook sirve como un pequeño "proof of concept", donde se implementa el experimento detallado en Frankle & Carbin sobre una variación de LeNet. Se busca establecer una estructura general para los demás notebooks y arquitecturas más complejas, al igual que generar un pequeño baseline para las partes comparativas del proyecto.

Imports - Setup

In [ ]:
```
import marimo as mo
import torch
from torch.utils.data import Dataset
from torch import nn
from torchvision import datasets
from torchvision.transforms import v2
import matplotlib.pyplot as plt

# Set GPU - CPU as device
device = torch.accelerator.current_accelerator().type if
torch.accelerator.is_available() else "cpu"
print(f"Using {device} device")
```

```
Using cuda device
```

Dataset

Para este experimento estaremos utilizando el dataset de MNIST, elegido por ser un dataset clásico y además fácil de acceder con los datasets de torch.

Empezamos cargando los conjuntos de prueba y entrenamiento, asegurándonos que las imágenes sean cargadas como tensores para cada uno.

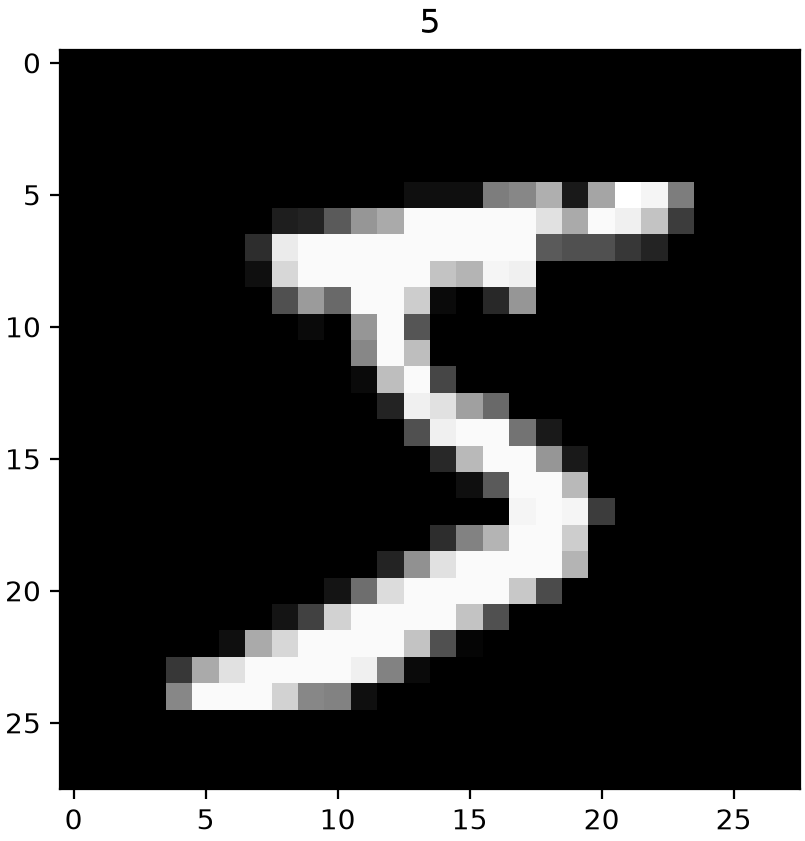
In [ ]:
```
# Datos de entrenamiento
mnist_train = datasets.MNIST(
    root="data",
    train=True,
    # Descargar si no está presente en el directorio "data"
    download=True,
```

```
    # Imagen PIL -> Tensor -> Tensor de Float32
    transform = v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32,
scale=True)])
)

# Datos de prueba
mnist_test = datasets.MNIST(
    root="data",
    train=False,
    # Descargar si no está presente en el directorio "data"
    download=True,
    # Imagen PIL -> Tensor -> Tensor de Float32
    transform = v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32,
scale=True)])
)
```

Muestra del Conjunto de Entrenamiento

In [ ]:
```
img, label = mnist_train[0]
fig, ax = plt.subplots()
ax.imshow(img.squeeze(), cmap="gray")
ax.set_title(label)
ax
```



Data Loader

In [ ]:
```python
from torch.utils.data import DataLoader

# Batch size de 60 (paper original de LTH)
train_loader = DataLoader(mnist_train, 60, shuffle=True)
test_loader = DataLoader(mnist_test, 60, shuffle=False)

x, y = next(iter(train_loader))
print(f"Feature batch shape: {x.shape}")
print(f"Label batch shape: {y.shape}")
```

```
Feature batch shape: torch.Size([60, 1, 28, 28])
Label batch shape: torch.Size([60])
```

MLP

Este MLP sigue la siguiente arquitectura:

- Input: 28x28 = 784
- Capa Densa: 784 -> 300 -> ReLU
- Capa Densa: 300 -> 100 -> ReLU
- Capa Densa: 100 -> 10

In [ ]:
```python
class MLP(nn.Module):
    def __init__(self):
        super().__init__()

        # Aplanar de 28x28 a 784
        self.flatten = nn.Flatten()

        # Capas densas
        self.layers = nn.Sequential(
            nn.Linear(784, 300),
            nn.ReLU(),
            nn.Linear(300, 100),
            nn.ReLU(),
            nn.Linear(100, 10),
        )

    # Forward: Aplana y pasa por capas densas
    def forward(self, x):
        x = self.flatten(x)
        logits = self.layers(x)
        # Se retornan los logits
        return logits

# Instanciar modelo
model = MLP()

# Pasar a GPU
model.to(device)

# Inspeccionar
model
```

```
MLP                                                    266.6K params · 1.0 MB

  flatten  Flatten  start_dim=1, end_dim=-1
▸ layers   Sequential                                          266.6K
```

Podemos confirmar que el modelo fue creado exitosamente, por lo que podemos pasar a la parte de entrenamiento.

Entrenamiento

Iniciamos primero almacenando una copia de los pesos iniciales antes del entrenamiento.

In [ ]:
```python
import copy

init_state = copy.deepcopy(model.state_dict())
```

Utilizamos entropía cruzada como función de pérdida y Adam como optimizador.

In [ ]:
```python
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr = 1.2e-3)
```

Loop de Entrenamiento

Aquí definimos algunas funciones para uso futuro y corremos un pequeño loop para verificar que el MLP si logre realizar predicciones adecuadas.

In [ ]:
```python
# Función de una época de entrenamiento
def train(model, loader, loss_fn, optimizer, masks=None):
    # Colocar el modelo en modo entrenamiento
    model.train()
    # Para cada entrada del loader
    for x, y in loader:
        # Pasar a GPU
        x, y = x.to(device), y.to(device)
        # Logits del forward
        logits = model(x)
        # Calcular pérdida
        loss = loss_fn(logits, y)
        # Reiniciar gradiente
        optimizer.zero_grad()
        # Backward pass
        loss.backward()
        # Paso de optimizador
        optimizer.step()
        # Para 'filtrar'
        if masks is not None:
```

```
            apply_masks(model, masks)

# Función de evaluación en conjunto de prueba
def evaluate(model, loader):
    # Colocar el modelo en modo evaluación
    model.eval()
    # Predicciones correctas
    correct = 0
    # No almacenar gradiente
    with torch.no_grad():
        # Para cada entrada del loader
        for x, y in loader:
            x, y = x.to(device), y.to(device)
            # Predicción = mayor logit
            preds = model(x).argmax(dim=1)
            correct += (preds == y).sum().item()
    # Retorna % de correctos
    return correct / len(loader.dataset)

# Loop de entrenamiento por 5 épocas
for epoch in range(5):
    train(model, train_loader, loss_fn, optimizer)
    acc = evaluate(model, test_loader)
    print(f"Epoch {epoch + 1}: test acc = {acc:.4f}")
```

```
Epoch 1: test acc = 0.9643
Epoch 2: test acc = 0.9700
Epoch 3: test acc = 0.9737
Epoch 4: test acc = 0.9781
Epoch 5: test acc = 0.9781
```

Podemos ver que se alcanza alrededor del 98% de accurady, lo cuál es bastante bueno y alrededor de lo esperado utilizando una arquitectura simple.

Pruning

En esta sección se implementan algunas funciones para poder realizar el pruning, buscamos mantener diferentes porcentajes de los pesos iniciales y comparar un entrenamiento aleatorio vs con los pesos iniciales. De esta manera identificar los "winning tickets".

In [ ]:
```
for name, p in model.named_parameters():
    print(name, p.shape)
```

```
layers.0.weight torch.Size([300, 784])
layers.0.bias torch.Size([300])
layers.2.weight torch.Size([100, 300])
layers.2.bias torch.Size([100])
layers.4.weight torch.Size([10, 100])
layers.4.bias torch.Size([10])
```

In [ ]:
```
# Returns a tensor of 1's with parameter shape, for each parameter in the
MLP
def make_masks(model):
```

```
    return {name: torch.ones_like(p) for name, p in
model.named_parameters() if "weight" in name}
```

In [ ]:
```
# Applies masks, keeps values the same if mask = 1 else reset to 0.
def apply_masks(model, masks):
    with torch.no_grad():
        for name, p in model.named_parameters():
            if name in masks:
                # mul_ mutates the parameter in place
                p.mul_(masks[name])
```

In [ ]:
```
# Creates masks and applies filter to prune out bottom 'rate' % of weight
magnitude
def prune(model, masks, rate):
    new_masks = {}
    for name, p in model.named_parameters():
        if name not in masks:
            continue

        mask = masks[name]

        # Flat 1D tensor of the surviving weight's magnitudes
        survivors = p[mask == 1].abs()

        k = int(rate * survivors.numel())

        if k == 0:
            new_masks[name] = mask.clone()
            continue

        cutoff = survivors.kthvalue(k).values

        new_masks[name] = mask * (p.abs() > cutoff).float()

    return new_masks
```

In [ ]:
```
def train_model(model, masks, epochs, lr=1.2e-3):
    loss_fn = nn.CrossEntropyLoss()
    opt = torch.optim.Adam(model.parameters(), 1.2e-3)
    for _ in range(epochs):
        train(model, train_loader, loss_fn, opt, masks)
    return evaluate(model, test_loader)

def sparsity_report(masks):
    total = sum(m.numel() for m in masks.values())
    kept = sum(m.sum().item() for m in masks.values())
    return kept / total
```

In [ ]:
```
def run_imp(trained_model, init_state, rounds, epochs, rate=0.2):
    masks = make_masks(trained_model)
    acc = evaluate(trained_model, test_loader)
    results = [{"round" : 0, "weights_left": 1.0, "acc": acc}]
    all_masks = [masks]

    for r in range(1, rounds + 1):
```

```
        # Cut smallest 20% of survivors
        masks = prune(trained_model, masks, rate)
        # Rewind to original
        trained_model.load_state_dict(init_state)
        apply_masks(trained_model, masks)
        acc = train_model(trained_model, masks, epochs)

        left = sparsity_report(masks)
        results.append({"round": r, "weights_left": left, "acc": acc})
        all_masks.append(masks)
        print(f"Round {r}: {left:.1%} left, acc = {acc:.4f}")

    return results, all_masks

def run_random_control(all_masks, epochs):
    results = []
    for r, masks in enumerate(all_masks):
        m = MLP().to(device)
        apply_masks(m, masks)
        acc = train_model(m, masks, epochs)

        left = sparsity_report(masks)
        results.append({"round": r, "weights_left": left, "acc": acc})
        print(f"Round {r}: {left:.1%} left, acc = {acc:.4f}")

    return results
```

In [ ]:

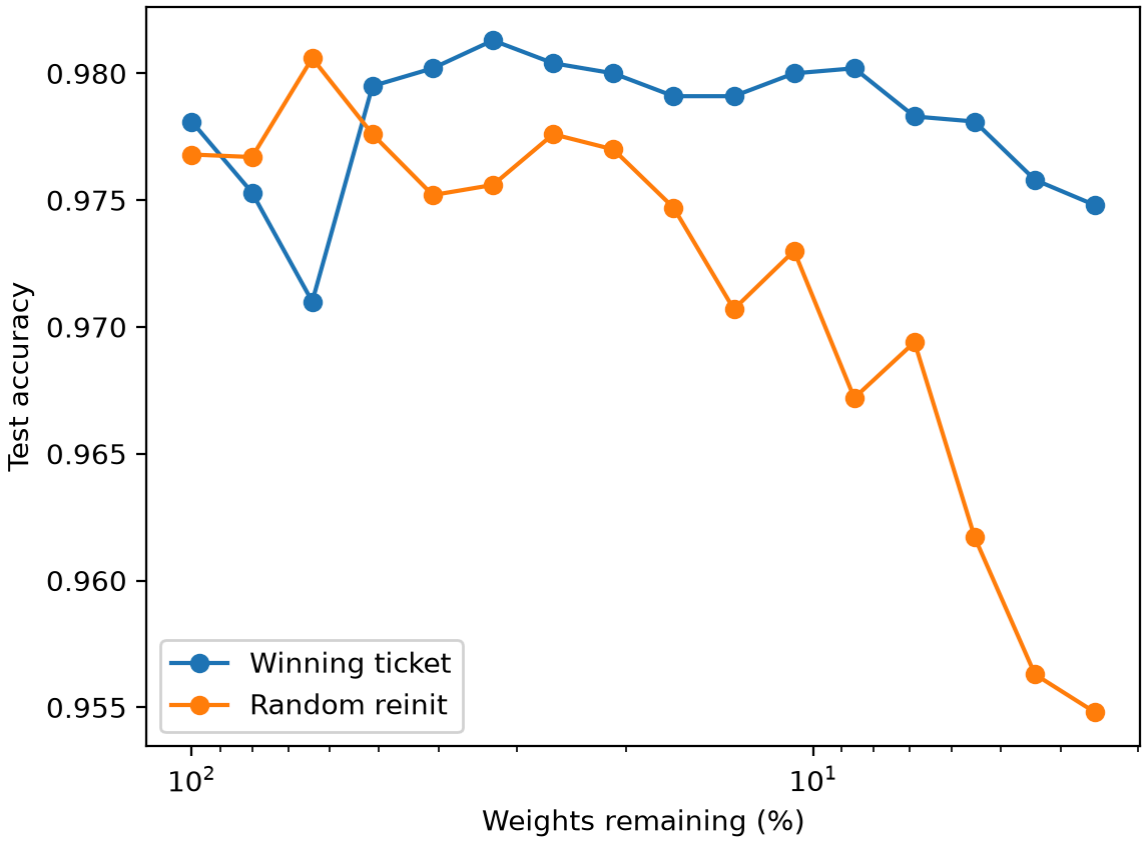
```
ticket_results, ticket_masks = run_imp(model, init_state, rounds=15,
epochs=5)

random_results = run_random_control(ticket_masks, epochs=5)
```

```
Round 1: 80.0% left, acc = 0.9753
Round 2: 64.0% left, acc = 0.9710
Round 3: 51.2% left, acc = 0.9795
Round 4: 41.0% left, acc = 0.9802
Round 5: 32.8% left, acc = 0.9813
Round 6: 26.2% left, acc = 0.9804
Round 7: 21.0% left, acc = 0.9800
Round 8: 16.8% left, acc = 0.9791
Round 9: 13.4% left, acc = 0.9791
Round 10: 10.7% left, acc = 0.9800
Round 11: 8.6% left, acc = 0.9802
Round 12: 6.9% left, acc = 0.9783
Round 13: 5.5% left, acc = 0.9781
Round 14: 4.4% left, acc = 0.9758
Round 15: 3.5% left, acc = 0.9748
Round 0: 100.0% left, acc = 0.9768
Round 1: 80.0% left, acc = 0.9767
Round 2: 64.0% left, acc = 0.9806
Round 3: 51.2% left, acc = 0.9776
Round 4: 41.0% left, acc = 0.9752
Round 5: 32.8% left, acc = 0.9756
Round 6: 26.2% left, acc = 0.9776
Round 7: 21.0% left, acc = 0.9770
Round 8: 16.8% left, acc = 0.9747
Round 9: 13.4% left, acc = 0.9707
Round 10: 10.7% left, acc = 0.9730
Round 11: 8.6% left, acc = 0.9672
Round 12: 6.9% left, acc = 0.9694
Round 13: 5.5% left, acc = 0.9617
Round 14: 4.4% left, acc = 0.9563
Round 15: 3.5% left, acc = 0.9548
```

In [ ]:

```
_fig, _ax = plt.subplots()
_ax.plot([r["weights_left"] * 100 for r in ticket_results],
         [r["acc"] for r in ticket_results], marker="o", label="Winning
ticket")
_ax.plot([r["weights_left"] * 100 for r in random_results],
         [r["acc"] for r in random_results], marker="o", label="Random
reinit")
_ax.set_xscale("log")
_ax.invert_xaxis()
_ax.set_xlabel("Weights remaining (%)")
_ax.set_ylabel("Test accuracy")
_ax.legend()
_ax
```


0.980
0.975
0.970
0.965
0.960
0.955
Test accuracy
Winning ticket
Random reinit
10^2
10^1
Weights remaining (%)